OpenClaw in meinem Alltag

Vom Chatbot zum persönlichen Ops-Agenten

Nicht nur **antworten**, sondern **Kontext ziehen, priorisieren, erinnern und handeln**.

Meta-Pointe: Dieses Deck wurde selbst mit **OpenClaw** erstellt — und zwar direkt über das **OpenClaw TUI** im laufenden System.

Claus Käpplinger · Kurzvorstellung / ZeitgAIst-Runde

Das eigentliche Problem

Nicht zu wenig AI.

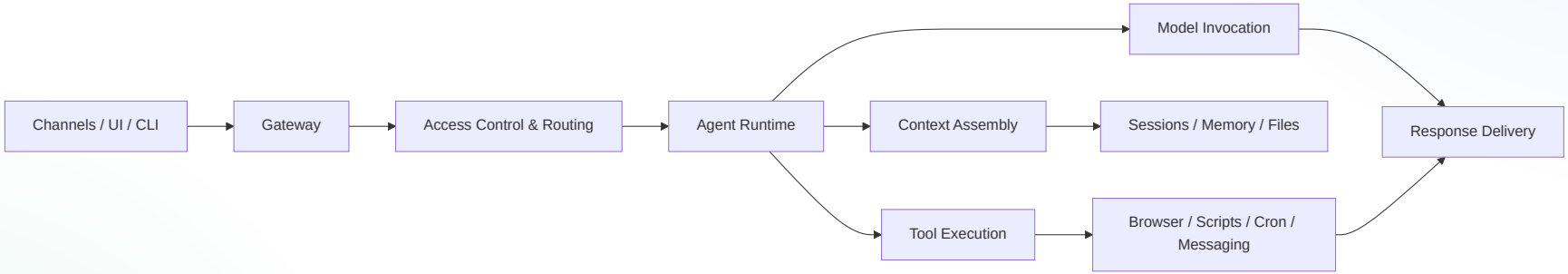
Zu viele offene Schleifen in zu vielen Kanälen.

Meine These

OpenClaw wird spannend, wenn es
zwischen Kanälen und Handlung sitzt.

Also nicht nur: „Prompt rein, Text raus“
sondern: **Chat + lokale Daten + Tools + proaktive Ausführung**

Architektur-Idee dahinter



Nicht nur Personal Assistant

Auch als Multi-Agent-Team denkbar

- dieselbe Runtime kann auch **mehrere spezialisierte Agenten** koordinieren
- z. B. Strategie, Dev, Marketing, Business mit **geteiltem Kontext**
- ein gemeinsamer Chat fungiert dann als **Control Plane**
- für mich spannend: **gleiche Plattform, anderer Organisationsmodus**

> Heute zeige ich die **Personal-Assistant-Variante** — aber das Modell skaliert auch in Richtung **Agent-Team**.

Beispiel 1

Tagessteuerung über WhatsApp

- Ich kann einfach schreiben:

  „Was ist heute wichtig?“
- OpenClaw zieht dazu **lokalen Kalender-Kontext**
- bei mir läuft das über **vdirsyncer + khal**
- Ziel ist nicht nur Terminanzeige, sondern: **ein echter Fokus für den Tag**

Was ist heute wichtig?

14:00 Hochschulwahlen mit IT
15:00 ZeitgAlst: Open Claw
20:30 Kennenlerntermin THW

Dazwischen nur ein echtes Arbeitsziel, nicht fünf.

Quelle (eigene Praxis): lokaler Kalender via **vdirsyncer/khal**; Fokus auf echte Commitments.

> Nicht: „Hier sind deine Termine.“
> Sondern eher: „Du hast 3 feste Blöcke — dazwischen genau ein sinnvolles Arbeitsziel.“

Beispiel 2

Proaktive Assistenz auf WhatsApp

- tägliche **Check-ins**, **Shutdowns** und **Reminder**
- nicht nur auf Anfrage, sondern auch **zum richtigen Zeitpunkt**
- mit Guardrails: dedizierter Bot interaktiv, Hauptkanal kontrollierter
- dadurch wird AI zu einer **Assistenzschicht im Alltag**

> Das ist für mich der eigentliche Sprung:
> **von reaktivem Chat zu leichter proaktiver Handlungsunterstützung.**

Daily check-in. Reply with 4 lines...

Bedtime ramp (15 min). Goal: devices OFF at 21:00.

Kurz-Check: Was machst du gerade? Bist du auf deinem BIG 1?

Quelle (eigene Praxis): WhatsApp-Rollenmodell + tägliche Cron-/Heartbeat-Check-ins.

Beispiel 3

Mail-Triage mit lokaler Pipeline

- Für Mail-Triage gibt es einen **lokalen Runner**
- Ergebnisse landen als **Reports**, nicht in einer Blackbox
- Relevante Mails werden gefiltert und kurz zurückgespielt
- Ziel: **weniger Overwhelm und weniger Kontextwechsel**

Konkret im Setup:

```
~/openclaw/scripts/mail_triage_runner.py
~/openclaw/reports/mail-triage/
```

> Nicht: „AI liest halt Mails.“
> Sondern: **Ich sehe schneller, worauf ich reagieren muss.**

Die Meta-Ebene: Dieses Deck wurde mit OpenClaw gebaut

> Die schönste Demo ist fast die Meta-Demo: OpenClaw erklärt hier nicht nur sich selbst — es war auch das Werkzeug, mit dem diese Präsentation über das TUI gebaut, exportiert und veröffentlicht wurde.

Meine Narrative in einem Satz

OpenClaw ist für mich kein weiterer smarter Chatbot, sondern **eine persönliche Runtime, die zwischen meinen Kanälen und meinen nächsten sinnvollen Handlungen sitzt**.

Danke

Wenn ihr wollt, zeige ich danach kurz den WhatsApp-/TUI-/Workflow-Teil live.